Programación III
Presentación de Cátedra
Introduccion

Alejandro Sánchez
aljsanchez@gmail.com

Departamento de Informática
Universidad Nacional de San Luis

San Luis

Programación III
Presentación de Cátedra – Intro

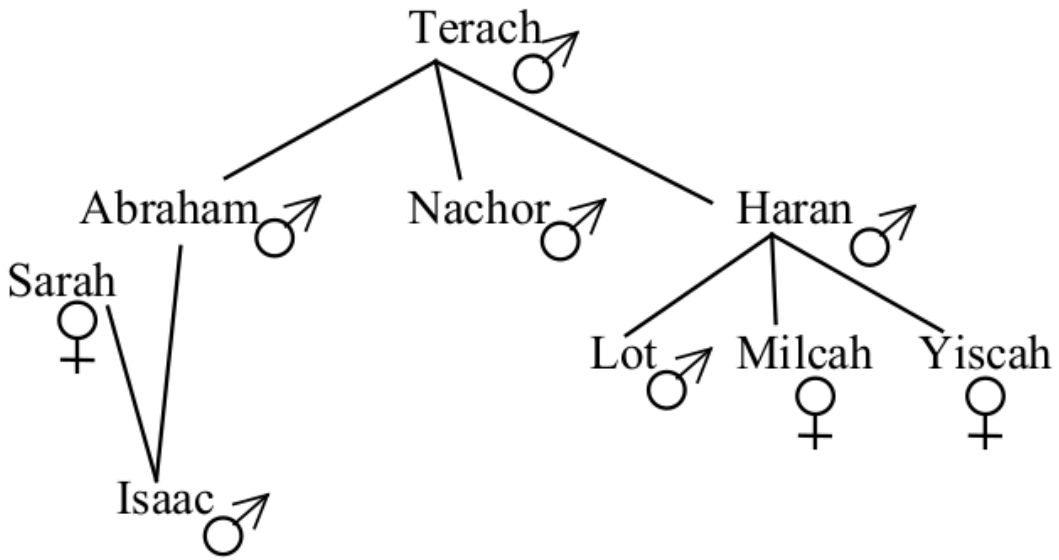
1. Introducción
2. Claúsulas de Horn
3. Términos
4. Operadores
5. Ejercicio
6. Modos de funcionamiento
7. Listas

Contexto histórico

- PROLOG del francés *PROgrammation en LOGique*
- Primer lenguaje del paradigma lógico
  Basado en Lógica de Primer Orden
- Su primera implementación fue en los años 70
  - Grupo dirigido por Alain Colmeraeur
    Universidad de Marsella
  - Utiliza resultados de Robert Kowalski
    Universidad de Edimburgo
- En 1995 se define su estándar ISO

El paradigma de PROLOG

- Declarativo
  Se especifica **que** se tiene que hacer
  en lugar de **como** se debe hacer
- Interpretado
- Representar e inferir conocimiento sobre un dominio
  - Declarar **hechos** sobre objetos y sus relaciones
  - Definir **reglas** sobre esos objetos y relaciones
  - Hacer **consultas**



Interprete



https://www.swi-prolog.org/

- Interprete gratuito y open source
- Respeta estándar ISO
- Versiones para: Linux, Windows, OSX
- Hay otros

Programación III
Presentación de Cátedra – Intro

1. Introducción
2. Claúsulas de Horn
3. Términos
4. Operadores
5. Ejercicio
6. Modos de funcionamiento
7. Listas

Programa en PROLOG

- Es un conjunto de **claúsulas de Horn**
- $p_1(\ldots) \wedge p_2(\ldots) \wedge \ldots \wedge p_m(\ldots) \Rightarrow p(t_1, t_2, \ldots, t_n)$, $m \geq 0$
- Símbolos de **Predicados**: $p, p_1, \ldots p_m$
- **Términos**: $t_1, t_2, \ldots, t_n$
- PROLOG: $\mathtt{p(t_1, t_2, \ldots, t_n)} \mathtt{:-} \mathtt{p_1(\ldots), p_2(\ldots), \ldots, p_m(\ldots).}$
  - Se usa $\mathtt{:-}$ en lugar de $\Rightarrow$
  - Deben terminar en punto
- Constructores de PROLOG
  - **Hechos**: si $m = 0$
    - Propiedad: si $n = 1$
    - Relación: si $n > 1$
  - **Reglas**: si $m > 0$
  - **Consultas**: Sin cabeza (sin $p(t_1, t_2, \ldots, t_n)$)
    (también se llama claúsula negativa)

Clasificación de animales

Hechos

H1: El animal es mamífero.

H2: El animal tiene pezuñas.

H3: El animal tiene rayas negras.

Reglas

R1: Si un animal es ungulado y tiene rayas negras
$\Rightarrow$ es una cebra.

R2: Si un animal rumia y es mamífero
$\Rightarrow$ es ungulado.

R3: Si un animal es mamífero y tiene pezuñas
$\Rightarrow$ es ungulado.

¿Es el animal una cebra?

Clasificación de animales: Programa PROLOG

Reglas y Hechos

```
% H1
es_mamífero.
% H2
tiene_pezuñas.
% H3
tiene_rayas_negras.

% R1
es_cebra :-es_ungulado, tiene_rayas_negras.
% R2
es_ungulado :-rumia, es_mamífero.
% R3
es_ungulado :-es_mamífero, tiene_pezuñas.

% La siguiente regla es para evitar error
% por predicado no definido
:-set_prolog_flag(unknown,fail).
```

Clasificación de animales: SWI Prolog

```
→  prolog git:(master) ✗ swipl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.5)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

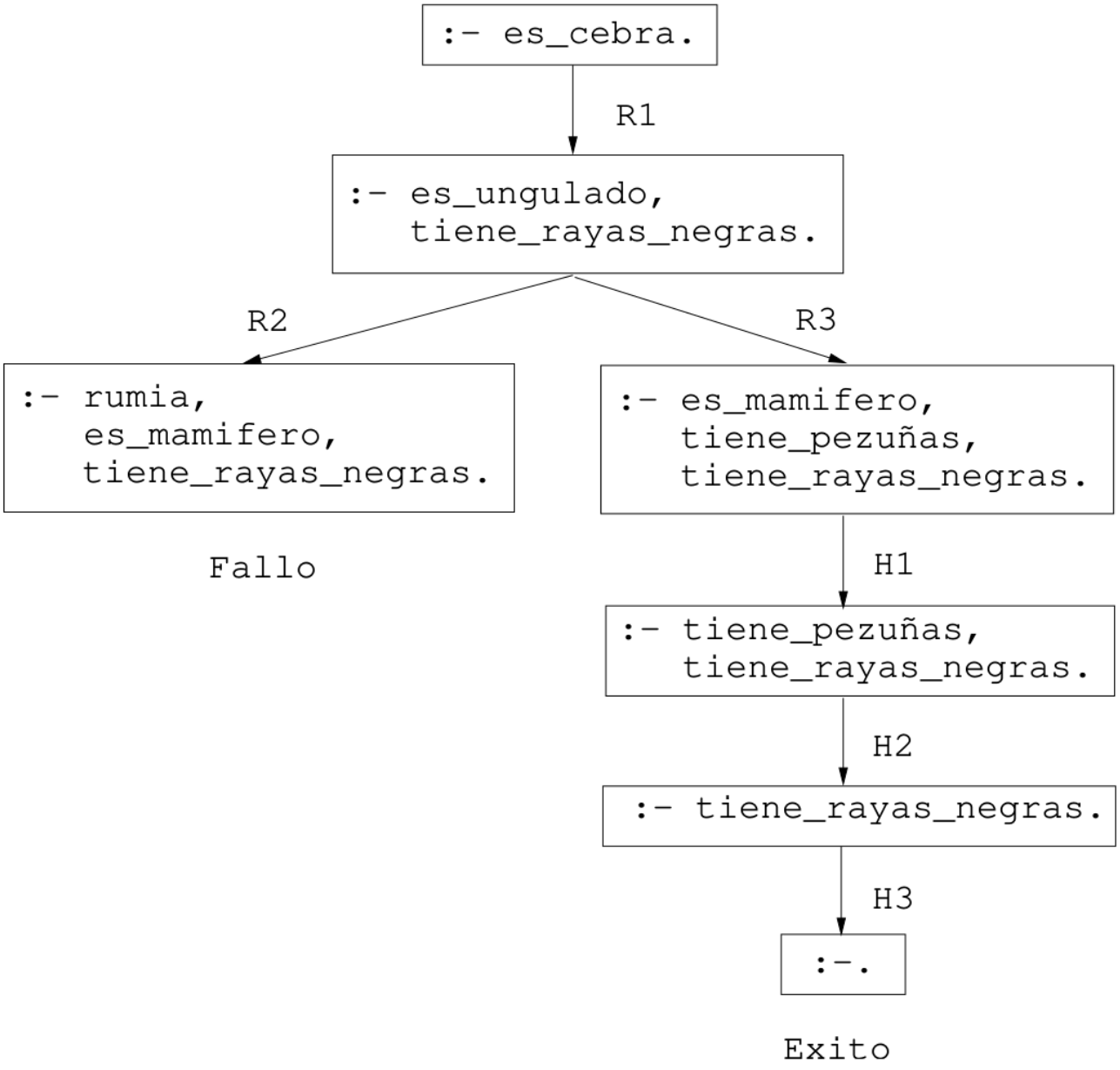
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- [cebra].
Warning: /home/alejandro/repos/teach/prog3/tex-source/unidades/prolog/0
1-Prolog/slide/prolog/cebra.pl:16:
Warning:    Using a non-error value for unknown in the global module
Warning:    causes most of the development environment to stop working.
Warning:    Please use :- dynamic or limit usage of unknown to a module
.
Warning:    See https://www.swi-prolog.org/howto/database.html
true.

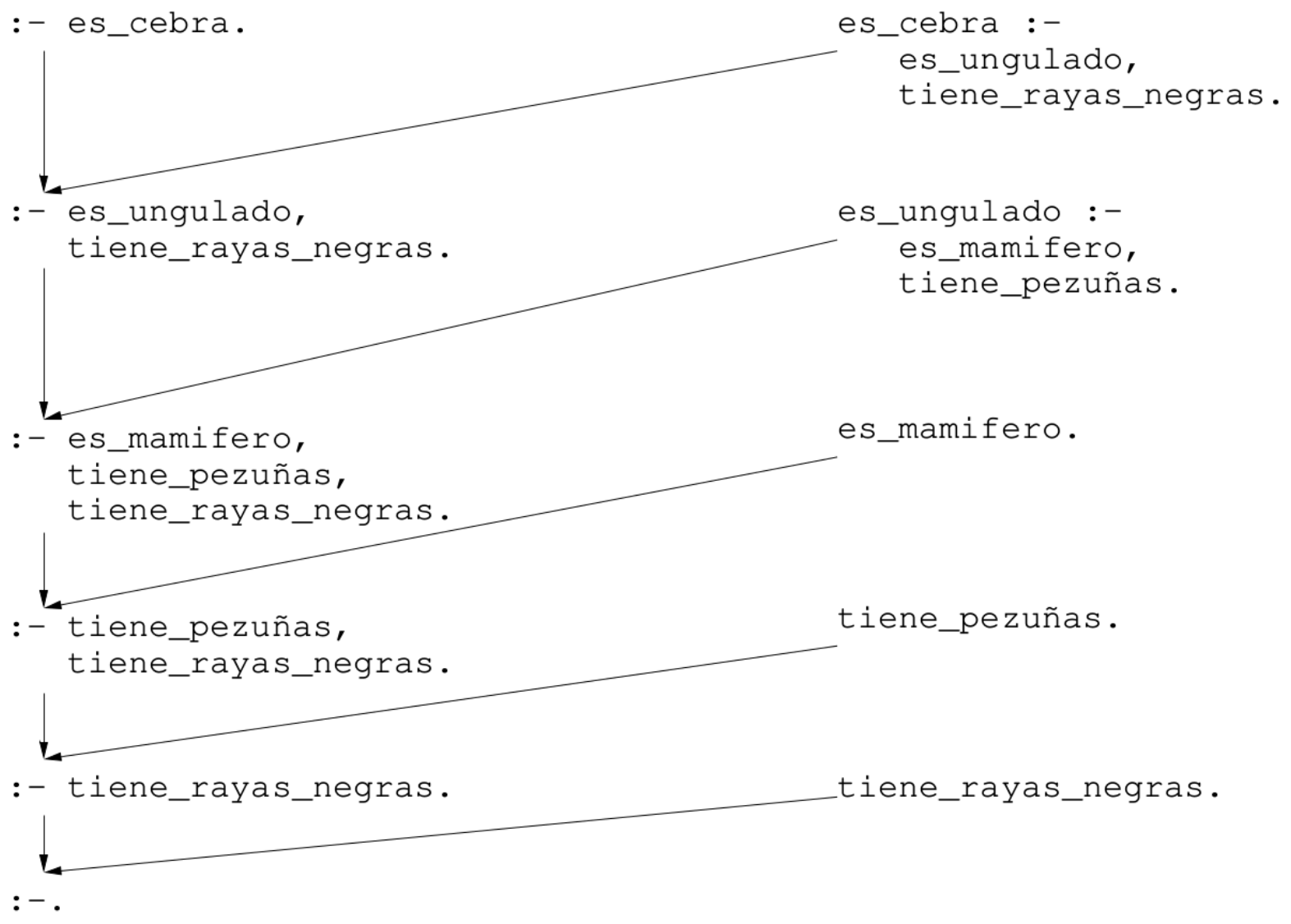
?- es_cebra.
true.

?- halt.
→  prolog git:(master) ✗
```

Clasificación de animales: Árbol de búsqueda



Clasificación de animales: Unificación y resolución



Programación III
Presentación de Cátedra – Intro

1. Introducción
2. Claúsulas de Horn
3. Términos
4. Operadores
5. Ejercicio
6. Modos de funcionamiento
7. Listas

Términos

- Argumentos/componentes de predicados/estructuras
- Es una secuencia de caracteres
- Categorías de caracteres para su sintaxis
  - Mayúsculas
  - Minúsculas
  - Dígitos 0-9
  - Símbolos
- Un término puede ser una
  - constante
  - variable
  - estructura

Constantes

- Átomo: en minúsculas
  Nombres de
  - Objetos: `luis`, `eva`, `'Hola mundo'`, ...
  - Propiedades: `padre`, `hijo`, ...
  - Relaciones: `padre_de`
- Número
  - Entero: `77`, `93`, ...
  - Real: `3.14`, `1.618`, ...

Variables

- Representan objetos del dominio
- Tipos
  - Normales: objetos cuya identidad es importante
    Comienzan con mayúscula o '_'
    `X`, `Y`, ... o `_X`, `_Y`, ...
  - Anónimas: objetos cuya identidad no es importante
    `_`
- Está **instanciada** si representa un objeto determinado
- **No** está **instanciada** si aún no está ligada a un valor

Variables: Cuantificación implícita

- En hechos: $\forall$

  $\texttt{conoce}(\texttt{profe}, \texttt{X}).$

  $\overline{\forall\, \texttt{x}\ \texttt{conoce}(\texttt{profe}, \texttt{x})}$

- En reglas
  - En la cabeza: $\forall$
  - En el cuerpo pero no en la cabeza: $\exists$

  $\texttt{abuelo}(\texttt{X}, \texttt{Y})\ \texttt{:-}\ \texttt{padre}(\texttt{X}, \texttt{Z}), \texttt{padre}(\texttt{Z}, \texttt{Y}).$

  $\overline{\forall\, \texttt{x}\ \forall\, \texttt{y}\ (\exists\, \texttt{z}\ \texttt{padre}(\texttt{x}, \texttt{z}) \wedge \texttt{padre}(\texttt{z}, \texttt{y})) \rightarrow \texttt{abuelo}(\texttt{x}, \texttt{y})}$

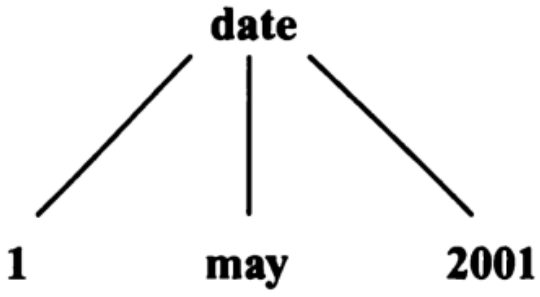
- En consultas: $\neg\exists$ (negamos lo que se quiere demostrar)

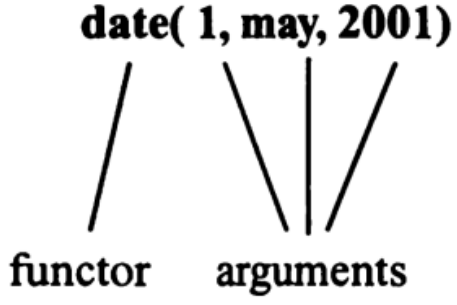
  $\texttt{?-}\ \texttt{conoce}(\texttt{profe}, \texttt{X}).$

  $\overline{\neg\exists\, \texttt{x}\ \texttt{conoce}(\texttt{profe}, \texttt{x})}$

Estructuras

- Son términos compuesto de otros términos
- Representan objetos que tienen varios componentes
- Los componentes se combinan con un **functor**
- Un functor identifica por
  - Un nombre
  - Una aridad
- Se pueden representar como árboles
- Ej.: `padre_de(luis, eva)`,`date(1, may, 2001)`





Programación III
Presentación de Cátedra – Intro

1. Introducción
2. Claúsulas de Horn
3. Términos
4. Operadores
5. Ejercicio
6. Modos de funcionamiento
7. Listas

Tipos de operadores de igualdad

- Igualdad aritmética '=:='
  `igual1(X,Y):-X =:= Y.`
- Identidad '=='
  `igual2(X,Y):-X == Y.`
- Unificación (matching) '='
  `igual3(X,Y):-X = Y.`
  `igual4(X,X).`
- Asignación '`is`'
  `igual5(X,Y):-X is Y.`

Tipos de operadores de igualdad: Ejemplos

Operator	=:=	==	=	=	**is**
Ejemplo	igual1	igual2	igual3	igual4	igual5
Igual?(3,3).	yes	yes	yes	yes	yes
Igual?(1,3).	no	no	no	no	no
Igual?(1+2,3).	yes	no	no	no	no
Igual?(3,1+2).	yes	no	no	no	yes
Igual?(X,1+2).	*	no	X=1+2	X=1+2	X=3
Igual?(1+2,X).	*	no	X=1+2	X=1+2	*
Igual?(3,X).	*	no	X=3	X=3	*
Igual?(X,3).	*	no	X=3	X=3	X=3
Igual?('hola','hola').	**	yes	yes	yes	yes
Igual?(1.0,1).	yes	no	no	no	no***
Igual?(1,1.0).	yes	no	no	no	yes***
Igual?(1.0,1.0).	yes	yes	yes	yes	yes***
Igual?(1.1,1.1).	yes	yes	yes	yes	yes***

* Excepción: Variable X libre.

** Error: 'hola' no es función (o expresión) válida.

*** Tiende a convertir reales a enteros (segundo arg. de **is**)

Operadores aritméticos y relacionales

- Potencia: `^`
- Modulo: `mod`
- Multiplicación y división: `* /`
- Suma y resta: `+ -`
- 'Igualdad' y desigualdad: `=`, `\=`
- Menor (o igual) y Mayor (o igual) que: `<`, `<=`, `>`, `>=`

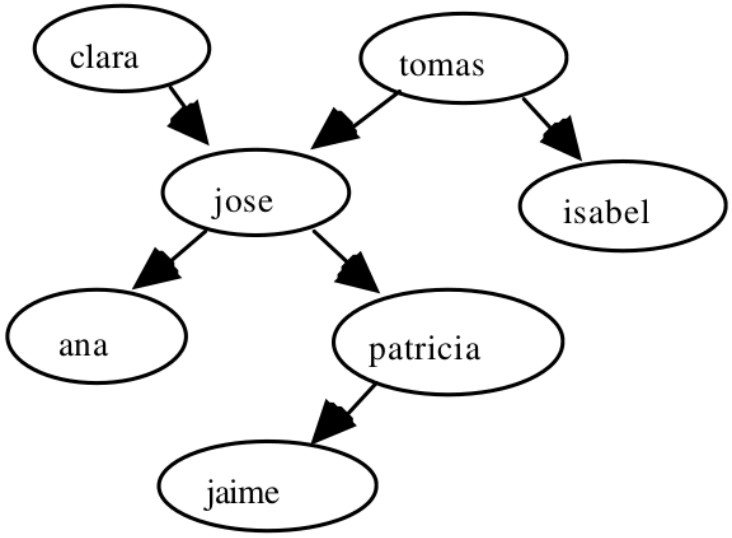
Ver apunte de aritmética en el aula virtual.

Programación III
Presentación de Cátedra – Intro

1. Introducción
2. Claúsulas de Horn
3. Términos
4. Operadores
5. Ejercicio
6. Modos de funcionamiento
7. Listas

Ejercicio

Dados los hechos:




```
progenitor(clara,jose).
progenitor(tomas,jose).
progenitor(tomas,isabel).
progenitor(jose,ana).
progenitor(jose,patricia).
progenitor(patricia,jaime).
dif(X,Y):-X\=Y.
```

- Agregar los hechos `hombre` y `mujer`
- Agregar las reglas
  - `es_madre(X)`
  - `es_padre(X)`
  - `es_hijo(X)`
  - `hermana_de(X, Y)`
  - `hermanos(X, Y)` (Una persona no es hermano propio)

Programación III
Presentación de Cátedra – Intro

1. Introducción
2. Claúsulas de Horn
3. Términos
4. Operadores
5. Ejercicio
6. Modos de funcionamiento
7. Listas

Modo de funcionamiento de los argumentos

- Entrada – representado con ‘$+$’
  - Constantes
  - Variables instanciadas
  - Estructuras
- Salida – representado con ‘$-$’
  - Variables libres
- Entrada/Salida – representado con ‘$?$’

Modo de funcionamiento de las reglas

- Combinación de los modos de los argumentos
- Una regla puede no soportar determinados modos

`igual1(X,Y):- X =:= Y.`
`igual1(+,+)` S
`igual1(-,+)` N
`igual1(+,-)` N
`igual1(-,-)` N

M: `igual1(+,+)`

`igual4(X,X).`
`igual4(+,+)` S
`igual4(-,+)` S
`igual4(+,-)` S
`igual4(-,-)` S

M: `igual4(?,?)`

`igual5(X,Y):- X is Y.`
`igual5(+,+)` S
`igual5(-,+)` S
`igual5(+,-)` N
`igual5(-,-)` N

M: `igual5(?,+)`

Programación III
Presentación de Cátedra – Intro

1. Introducción
2. Claúsulas de Horn
3. Términos
4. Operadores
5. Ejercicio
6. Modos de funcionamiento
7. Listas

Sintaxis

- Constructores
  - `[]` la lista vacía
  - `'[|]'(a,L)` con `L` una lista
- Sintaxis abreviada
  - $[a_1, \ldots, a_n]$
  - $[a_1|L]$

```
?- '[|]'(X,Y)=[a].
X = a,
Y = [].

?- '[|]'(X,Y)=[a,b].
X = a,
Y = [b].

?- '[|]'(X, '[|]'(Y,Z))=[a,b].
X = a,
Y = b,
Z = [].

?- [X|Y] = [a,b,c,d].
X = a,
Y = [b, c, d].

?- [X,Y|Z] = [a,b,c,d].
X = a,
Y = b,
Z = [c, d].
```

Biblioteca de manipulación de listas

- Algunos predicados ...
  - `member(?Elem, ?List)`
    Verdadero si `Elem` es un miembro de `List`.

```
?- member(X, [One]).
X = One.
```

  - `permutation(?Xs, ?Ys)`
    Verdadero si las Xs son permutaciones de las Ys.

```
?- permutation([1,2], [X,Y]).
X = 1,
Y = 2 ;
X = 2,
Y = 1 ;
false.
```

- Más en Biblioteca de manipulación de listas